REVISA — Pacote físico de repositório pronto para colar

Estrutura do monorepo

```
revisa-platform/
├── .env.example
├── .gitignore
├── Makefile
├── README.md
├── docker-compose.yml
├── apps/
│   └── api/
│       ├── Dockerfile
│       ├── alembic.ini
│       ├── pyproject.toml
│       ├── app/
│       │   ├── main.py
│       │   ├── core/
│       │   │   ├── settings.py
│       │   │   ├── database.py
│       │   │   ├── security.py
│       │   │   ├── auth.py
│       │   │   ├── permissions.py
│       │   │   ├── scope.py
│       │   │   ├── audit.py
│       │   │   └── startup.py
│       │   ├── shared/
│       │   │   └── audit.py
│       │   ├── api/
│       │   │   ├── deps/
│       │   │   │   ├── auth.py
│       │   │   │   ├── permissions.py
│       │   │   │   └── scope.py
│       │   │   └── v1/
│       │   │       ├── api.py
│       │   │       └── routers/
│       │   │           ├── __init__.py
│       │   │           ├── auth.py
│       │   │           ├── persons.py
│       │   │           ├── contacts_capture.py
│       │   │           ├── polos.py
│       │   │           ├── tasks.py
│       │   │           └── dashboards.py
│       │   ├── domain/
│       │   │   ├── iam/
```

```
│       │   │   │   ├── models.py
│       │   │   │   └── repository.py
│       │   │   ├── core/
│       │   │   │   ├── models.py
│       │   │   │   ├── schemas.py
│       │   │   │   ├── repository.py
│       │   │   │   └── service.py
│       │   │   ├── territory/
│       │   │   │   ├── models.py
│       │   │   │   ├── schemas.py
│       │   │   │   ├── repository.py
│       │   │   │   └── service.py
│       │   │   ├── polo/
│       │   │   │   ├── models.py
│       │   │   │   ├── schemas.py
│       │   │   │   ├── repository.py
│       │   │   │   └── service.py
│       │   │   ├── workflow/
│       │   │   │   ├── models.py
│       │   │   │   ├── schemas.py
│       │   │   │   ├── repository.py
│       │   │   │   └── service.py
│       │   │   └── analytics/
│       │   │       ├── repository.py
│       │   │       └── service.py
│       │   └── tests/
│       │       ├── conftest.py
│       │       └── integration/
│       │           ├── test_auth_login.py
│       │           ├── test_persons.py
│       │           ├── test_contacts_capture.py
│       │           ├── test_polos.py
│       │           ├── test_tasks.py
│       │           └── test_dashboards.py
│       ├── alembic/
│       │   ├── env.py
│       │   └── versions/
│       │       ├── 0001_create_iam_schema.py
│       │       ├── 0002_create_core_schema.py
│       │       ├── 0003_create_territory_schema.py
│       │       ├── 0004_create_polo_schema.py
│       │       ├── 0005_create_events_schema.py
│       │       ├── 0006_create_workflow_schema.py
│       │       ├── 0007_create_governance_schema.py
│       │       └── 0008_create_analytics_schema.py
│       └── scripts/
│           ├── seed_initial_data.py
│           └── bootstrap_admin.py
├── database/
│   └── seeds/
```

```
│       └── permissions_seed.sql
└── .github/
    └── workflows/
        └── api-ci.yml
```

Arquivos raiz

`.env.example`

```
POSTGRES_DB=revisa
POSTGRES_USER=revisa
POSTGRES_PASSWORD=revisa
DATABASE_URL=postgresql+psycopg://revisa:revisa@localhost:5432/revisa
TEST_DATABASE_URL=postgresql+psycopg://revisa:revisa@localhost:5432/
revisa_test
JWT_SECRET_KEY=change_me
JWT_REFRESH_SECRET_KEY=change_me_too
API_V1_PREFIX=/api/v1
```

`.gitignore`

```
__pycache__/
*.pyc
.env
.venv/
.pytest_cache/
.mypy_cache/
coverage/
htmlcov/
```

`Makefile`

```
up:
    docker compose up --build

down:
    docker compose down

migrate:
    cd apps/api && alembic upgrade head

seed:
    cd apps/api && python scripts/seed_initial_data.py
```

```
test:
    cd apps/api && pytest -q
```

`README.md`

```
# REVISA Platform

Backend FastAPI da plataforma REVISA.

## Requisitos
- Python 3.11+
- PostgreSQL 16

## Instalação rápida
```bash
cp .env.example .env
cd apps/api
python -m venv .venv
source .venv/bin/activate
pip install -e .
alembic upgrade head
python scripts/seed_initial_data.py
python scripts/bootstrap_admin.py
uvicorn app.main:app --reload
```

Testes

```
cd apps/api
pytest -q
```

```
### `docker-compose.yml`
```yaml
version: "3.9"
services:
  postgres:
    image: postgres:16
    environment:
      POSTGRES_DB: revisa
      POSTGRES_USER: revisa
      POSTGRES_PASSWORD: revisa
    ports:
      - "5432:5432"
```

Backend API

`apps/api/pyproject.toml`

```
[project]
name = "revisa-api"
version = "1.0.0"
description = "REVISA API"
requires-python = ">=3.11"
dependencies = [
  "fastapi>=0.115.0",
  "uvicorn[standard]>=0.30.0",
  "sqlalchemy>=2.0.0",
  "alembic>=1.13.0",
  "psycopg[binary]>=3.2.0",
  "pydantic>=2.8.0",
  "pydantic-settings>=2.3.0",
  "pyjwt>=2.8.0",
  "passlib[bcrypt]>=1.7.4",
  "python-multipart>=0.0.9",
  "pytest>=8.3.0",
  "httpx>=0.27.0"
]

[tool.pytest.ini_options]
testpaths = ["app/tests"]
pythonpath = ["."]

[build-system]
requires = ["setuptools>=68", "wheel"]
build-backend = "setuptools.build_meta"
```

`apps/api/Dockerfile`

```
FROM python:3.11-slim
WORKDIR /app
COPY pyproject.toml .
RUN pip install --no-cache-dir -e .
COPY . .
EXPOSE 8000
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

`apps/api/alembic.ini`

```
[alembic]
script_location = alembic
sqlalchemy.url = postgresql+psycopg://revisa:revisa@localhost:5432/revisa
```

```
[loggers]
keys = root,sqlalchemy,alembic

[handlers]
keys = console

[formatters]
keys = generic

[logger_root]
level = WARN
handlers = console

[logger_sqlalchemy]
level = WARN
handlers =
qualname = sqlalchemy.engine

[logger_alembic]
level = INFO
handlers = console
qualname = alembic

[handler_console]
class = StreamHandler
args = (sys.stderr,)
level = NOTSET
formatter = generic

[formatter_generic]
format = %(levelname)-5.5s [%(name)s] %(message)s
```

`apps/api/alembic/env.py`

```
from logging.config import fileConfig

from alembic import context
from sqlalchemy import engine_from_config, pool

config = context.config
if config.config_file_name is not None:
    fileConfig(config.config_file_name)

target_metadata = None


def run_migrations_offline() -> None:
    url = config.get_main_option("sqlalchemy.url")
    context.configure(url=url, literal_binds=True, compare_type=True)
    with context.begin_transaction():
```

```
        context.run_migrations()


def run_migrations_online() -> None:
    connectable = engine_from_config(
        config.get_section(config.config_ini_section),
        prefix="sqlalchemy.",
        poolclass=pool.NullPool,
    )
    with connectable.connect() as connection:
        context.configure(connection=connection, compare_type=True)
        with context.begin_transaction():
            context.run_migrations()


if context.is_offline_mode():
    run_migrations_offline()
else:
    run_migrations_online()
```

`apps/api/app/main.py`

```
from fastapi import FastAPI

from app.api.v1.api import api_router
from app.core.startup import configure_middlewares

app = FastAPI(title="REVISA Platform API", version="1.0.0")
configure_middlewares(app)
app.include_router(api_router, prefix="/api/v1")


@app.get("/health")
def health() -> dict[str, str]:
    return {"status": "ok"}
```

Core

`apps/api/app/core/settings.py`

```
from pydantic_settings import BaseSettings


class Settings(BaseSettings):
    database_url: str
    test_database_url: str | None = None
    jwt_secret_key: str
```

```
    jwt_refresh_secret_key: str
    api_v1_prefix: str = "/api/v1"

    class Config:
        env_file = ".env"
        extra = "ignore"


settings = Settings()
```

`apps/api/app/core/database.py`

```
from sqlalchemy import create_engine
from sqlalchemy.orm import DeclarativeBase, sessionmaker

from app.core.settings import settings


class Base(DeclarativeBase):
    pass


engine = create_engine(settings.database_url, future=True,
pool_pre_ping=True)
SessionLocal = sessionmaker(bind=engine, autoflush=False, autocommit=False,
future=True)


def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

`apps/api/app/core/security.py`

```
from datetime import datetime, timedelta, timezone
from typing import Any

import jwt
from passlib.context import CryptContext

from app.core.settings import settings

pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")


def hash_password(password: str) -> str:
```

```
    return pwd_context.hash(password)


def verify_password(password: str, password_hash: str) -> bool:
    return pwd_context.verify(password, password_hash)


def _encode_token(subject: str, secret: str, expires_delta: timedelta,
token_type: str) -> str:
    now = datetime.now(timezone.utc)
    payload: dict[str, Any] = {
        "sub": subject,
        "type": token_type,
        "iat": now,
        "exp": now + expires_delta,
    }
    return jwt.encode(payload, secret, algorithm="HS256")


def create_access_token(subject: str) -> str:
    return _encode_token(subject, settings.jwt_secret_key,
timedelta(minutes=30), "access")


def create_refresh_token(subject: str) -> str:
    return _encode_token(subject, settings.jwt_refresh_secret_key,
timedelta(days=7), "refresh")


def decode_access_token(token: str) -> dict[str, Any]:
    return jwt.decode(token, settings.jwt_secret_key, algorithms=["HS256"])
```

`apps/api/app/core/auth.py`

```
from dataclasses import dataclass
from datetime import datetime, timedelta, timezone
from hashlib import sha256

from app.core.security import create_access_token, create_refresh_token,
decode_access_token, verify_password
from app.domain.iam.models import RefreshToken
from app.domain.iam.repository import IAMRepository


@dataclass
class CurrentUser:
    id: str
    username: str
    email: str
    roles: set[str]
```

```
    permissions: set[str]
    scope_map: dict[str, set[str]]
    is_global_admin: bool


class AuthService:
    def __init__(self, repo: IAMRepository):
        self.repo = repo

    def login(self, username: str, password: str):
        user = self.repo.get_user_by_username(username)
        if not user or user.status != "ACTIVE":
            return None
        if not verify_password(password, user.password_hash):
            return None

        access_token = create_access_token(str(user.id))
        refresh_token = create_refresh_token(str(user.id))
        refresh_hash = sha256(refresh_token.encode()).hexdigest()
        self.repo.save_refresh_token(
            RefreshToken(
                user_id=user.id,
                token_hash=refresh_hash,
                expires_at=datetime.now(timezone.utc) + timedelta(days=7),
            )
        )
        return {
            "access_token": access_token,
            "refresh_token": refresh_token,
            "token_type": "Bearer",
            "expires_in": 1800,
        }

    def build_current_user(self, access_token: str) -> CurrentUser:
        payload = decode_access_token(access_token)
        user_id = payload["sub"]
        user = self.repo.get_user_by_id(user_id)
        if not user:
            raise ValueError("User not found")
        roles = set(self.repo.get_role_codes(user_id))
        permissions = set(self.repo.get_permission_codes(user_id))
        scopes = self.repo.get_scopes(user_id)
        scope_map: dict[str, set[str]] = {}
        for scope in scopes:
            scope_map.setdefault(scope.scope_type,
set()).add(str(scope.scope_ref_id))
        return CurrentUser(
            id=str(user.id),
            username=user.username,
            email=user.email,
            roles=roles,
```

```
            permissions=permissions,
            scope_map=scope_map,
            is_global_admin="ADM_GERAL_REVISA" in roles,
        )
```

`apps/api/app/core/permissions.py`

```
def has_permission(current_user, permission_code: str) -> bool:
    return current_user.is_global_admin or permission_code in
current_user.permissions
```

`apps/api/app/core/scope.py`

```
def in_scope(current_user, scope_type: str, scope_ref_id: str | None) ->
bool:
    if current_user.is_global_admin:
        return True
    if scope_ref_id is None:
        return True
    return scope_ref_id in current_user.scope_map.get(scope_type, set())
```

`apps/api/app/core/audit.py`

```
import json
from functools import wraps

from sqlalchemy.orm import Session

from app.shared.audit import write_audit_log


def audited_mutation(action: str, entity_schema: str, entity_name: str):
    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            result = func(*args, **kwargs)
            db: Session | None = kwargs.get("db")
            current_user = kwargs.get("current_user")
            entity_id = getattr(result, "id", None)
            if db is not None:
                write_audit_log(
                    db,
                    user_id=getattr(current_user, "id", None),
                    action=action,
                    entity_schema=entity_schema,
                    entity_name=entity_name,
                    entity_id=entity_id,
                    old_values_json=None,
```

```
                    new_values_json=json.dumps({"id": str(entity_id)}) if
entity_id else None,
                )
            return result
        return wrapper
    return decorator
```

apps/api/app/core/startup.py

```
from fastapi.middleware.cors import CORSMiddleware


def configure_middlewares(app):
    app.add_middleware(
        CORSMiddleware,
        allow_origins=["*"],
        allow_credentials=True,
        allow_methods=["*"],
        allow_headers=["*"],
    )
```

apps/api/app/shared/audit.py

```
from sqlalchemy import text
from sqlalchemy.orm import Session


def write_audit_log(
    db: Session,
    *,
    user_id,
    action: str,
    entity_schema: str,
    entity_name: str,
    entity_id,
    old_values_json=None,
    new_values_json=None,
):
    stmt = text(
        """
        insert into governance.audit_logs (
            user_id, action, entity_schema, entity_name, entity_id,
old_values_json, new_values_json
        ) values (
            :user_id, :action, :entity_schema, :entity_name, :entity_id,
cast(:old_values_json as jsonb), cast(:new_values_json as jsonb)
        )
        """
    )
```

```
    db.execute(
        stmt,
        {
            "user_id": user_id,
            "action": action,
            "entity_schema": entity_schema,
            "entity_name": entity_name,
            "entity_id": entity_id,
            "old_values_json": old_values_json,
            "new_values_json": new_values_json,
        },
    )
```

API deps

`apps/api/app/api/deps/auth.py`

```
from fastapi import Depends, HTTPException, status
from fastapi.security import HTTPAuthorizationCredentials, HTTPBearer
from sqlalchemy.orm import Session

from app.core.auth import AuthService
from app.core.database import get_db
from app.domain.iam.repository import IAMRepository

bearer_scheme = HTTPBearer(auto_error=True)


def get_current_user(
    credentials: HTTPAuthorizationCredentials = Depends(bearer_scheme),
    db: Session = Depends(get_db),
):
    try:
        return
AuthService(IAMRepository(db)).build_current_user(credentials.credentials)
    except Exception:
        raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED,
detail="Invalid or expired token")
```

`apps/api/app/api/deps/permissions.py`

```
from fastapi import Depends, HTTPException, status

from app.api.deps.auth import get_current_user
from app.core.permissions import has_permission
```

```
def require_permission(permission_code: str):
    def dependency(current_user = Depends(get_current_user)):
        if not has_permission(current_user, permission_code):
            raise HTTPException(status_code=status.HTTP_403_FORBIDDEN,
detail=f"Missing permission: {permission_code}")
        return current_user
    return dependency
```

`apps/api/app/api/deps/scope.py`

```
from fastapi import HTTPException, status

from app.core.scope import in_scope


def ensure_scope(current_user, scope_type: str, scope_ref_id: str | None):
    if not in_scope(current_user, scope_type, scope_ref_id):
        raise HTTPException(status_code=status.HTTP_403_FORBIDDEN,
detail=f"Out of scope: {scope_type}")
```

Routers

`apps/api/app/api/v1/api.py`

```
from fastapi import APIRouter

from app.api.v1.routers import auth, contacts_capture, dashboards, persons,
polos, tasks

api_router = APIRouter()
api_router.include_router(auth.router, prefix="/auth", tags=["Auth"])
api_router.include_router(persons.router, prefix="/persons",
tags=["Persons"])
api_router.include_router(contacts_capture.router, prefix="/contacts-
capture", tags=["ContactsCapture"])
api_router.include_router(polos.router, prefix="/polos", tags=["Polos"])
api_router.include_router(tasks.router, prefix="/tasks", tags=["Tasks"])
api_router.include_router(dashboards.router, prefix="", tags=["Dashboards"])
```

`apps/api/app/api/v1/routers/__init__.py`

```
# package marker
```

`apps/api/app/api/v1/routers/auth.py`

```
from fastapi import APIRouter, Depends, HTTPException, status
from pydantic import BaseModel
from sqlalchemy.orm import Session

from app.api.deps.auth import get_current_user
from app.core.auth import AuthService
from app.core.database import get_db
from app.domain.iam.repository import IAMRepository

router = APIRouter()


class LoginRequest(BaseModel):
    username: str
    password: str


@router.post("/login")
def login(payload: LoginRequest, db: Session = Depends(get_db)):
    result = AuthService(IAMRepository(db)).login(payload.username,
payload.password)
    if not result:
        raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED,
detail="Invalid credentials")
    db.commit()
    return result


@router.get("/me")
def me(current_user = Depends(get_current_user)):
    return {
        "id": current_user.id,
        "username": current_user.username,
        "email": current_user.email,
        "roles": sorted(list(current_user.roles)),
        "permissions": sorted(list(current_user.permissions)),
        "scopes": {k: sorted(list(v)) for k, v in
current_user.scope_map.items()},
    }
```

`apps/api/app/api/v1/routers/persons.py`

```
from fastapi import APIRouter, Depends
from sqlalchemy.orm import Session

from app.api.deps.permissions import require_permission
from app.core.database import get_db
```

```
from app.domain.core.repository import CoreRepository
from app.domain.core.schemas import PersonCreate, PersonOut
from app.domain.core.service import CoreService

router = APIRouter()


@router.get("", response_model=list[PersonOut])
def list_persons(
    current_user = Depends(require_permission("person.read")),
    db: Session = Depends(get_db),
):
    return CoreService(CoreRepository(db)).list_persons()


@router.post("", response_model=PersonOut, status_code=201)
def create_person(
    payload: PersonCreate,
    current_user = Depends(require_permission("person.create")),
    db: Session = Depends(get_db),
):
    result = CoreService(CoreRepository(db)).create_person(payload, db=db,
current_user=current_user)
    db.commit()
    return result
```

`apps/api/app/api/v1/routers/contacts_capture.py`

```
from fastapi import APIRouter, Depends
from sqlalchemy.orm import Session

from app.api.deps.permissions import require_permission
from app.core.database import get_db
from app.domain.territory.repository import TerritoryRepository
from app.domain.territory.schemas import ContactCaptureCreate,
ContactCaptureOut
from app.domain.territory.service import TerritoryService

router = APIRouter()


@router.get("", response_model=list[ContactCaptureOut])
def list_captures(
    current_user = Depends(require_permission("capture.read")),
    db: Session = Depends(get_db),
):
    return TerritoryService(TerritoryRepository(db)).list_captures()


@router.post("", response_model=ContactCaptureOut, status_code=201)
```

```
def create_capture(
    payload: ContactCaptureCreate,
    current_user = Depends(require_permission("capture.create")),
    db: Session = Depends(get_db),
):
    result =
TerritoryService(TerritoryRepository(db)).create_capture(current_user.id,
payload, db=db, current_user=current_user)
    db.commit()
    return result
```

apps/api/app/api/v1/routers/polos.py

```
from uuid import UUID

from fastapi import APIRouter, Depends
from sqlalchemy.orm import Session

from app.api.deps.permissions import require_permission
from app.api.deps.scope import ensure_scope
from app.core.database import get_db
from app.domain.polo.repository import PoloRepository
from app.domain.polo.schemas import AttendanceCreate, AttendanceOut,
PoloBeneficiarioCreate, PoloBeneficiarioOut
from app.domain.polo.service import PoloService

router = APIRouter()


@router.get("/{id}/beneficiarios", response_model=list[PoloBeneficiarioOut])
def list_beneficiarios(
    id: UUID,
    current_user = Depends(require_permission("polo.manage_beneficiary")),
    db: Session = Depends(get_db),
):
    ensure_scope(current_user, "POLO", str(id))
    return PoloService(PoloRepository(db)).list_beneficiarios(id)


@router.post("/{id}/beneficiarios", response_model=PoloBeneficiarioOut,
status_code=201)
def create_beneficiario(
    id: UUID,
    payload: PoloBeneficiarioCreate,
    current_user = Depends(require_permission("polo.manage_beneficiary")),
    db: Session = Depends(get_db),
):
    ensure_scope(current_user, "POLO", str(id))
    result = PoloService(PoloRepository(db)).create_beneficiario(id,
payload, db=db, current_user=current_user)
```

```
    db.commit()
    return result


@router.post("/{id}/frequencias", response_model=AttendanceOut,
status_code=201)
def register_attendance(
    id: UUID,
    payload: AttendanceCreate,
    current_user = Depends(require_permission("attendance.create")),
    db: Session = Depends(get_db),
):
    ensure_scope(current_user, "POLO", str(id))
    result =
PoloService(PoloRepository(db)).register_attendance(current_user.id,
payload, db=db, current_user=current_user)
    db.commit()
    return result
```

`apps/api/app/api/v1/routers/tasks.py`

```
from fastapi import APIRouter, Depends
from sqlalchemy.orm import Session

from app.api.deps.permissions import require_permission
from app.core.database import get_db
from app.domain.workflow.repository import WorkflowRepository
from app.domain.workflow.schemas import TaskCreate, TaskOut
from app.domain.workflow.service import WorkflowService

router = APIRouter()


@router.get("", response_model=list[TaskOut])
def list_tasks(
    current_user = Depends(require_permission("task.read")),
    db: Session = Depends(get_db),
):
    return WorkflowService(WorkflowRepository(db)).list_tasks()


@router.post("", response_model=TaskOut, status_code=201)
def create_task(
    payload: TaskCreate,
    current_user = Depends(require_permission("task.create")),
    db: Session = Depends(get_db),
):
    result =
WorkflowService(WorkflowRepository(db)).create_task(current_user.id,
payload, db=db, current_user=current_user)
```

```
    db.commit()
    return result
```

`apps/api/app/api/v1/routers/dashboards.py`

```
from uuid import UUID

from fastapi import APIRouter, Depends, HTTPException
from sqlalchemy.orm import Session

from app.api.deps.permissions import require_permission
from app.core.database import get_db
from app.domain.analytics.repository import AnalyticsRepository
from app.domain.analytics.service import AnalyticsService

router = APIRouter()


@router.get("/vereadores/{id}/dashboard")
def vereador_dashboard(
    id: UUID,
    current_user = Depends(require_permission("dashboard.vereador.read")),
    db: Session = Depends(get_db),
):
    data =
AnalyticsService(AnalyticsRepository(db)).get_vereador_dashboard(id)
    if not data:
        raise HTTPException(status_code=404, detail="Dashboard não
encontrado")
    return data
```

Domínio IAM

`apps/api/app/domain/iam/models.py`

```
from datetime import datetime
from uuid import uuid4

from sqlalchemy import Boolean, DateTime, ForeignKey, String, func
from sqlalchemy.dialects.postgresql import UUID
from sqlalchemy.orm import Mapped, mapped_column

from app.core.database import Base


class User(Base):
    __tablename__ = "users"
```

```
    __table_args__ = {"schema": "iam"}

    id: Mapped[str] = mapped_column(UUID(as_uuid=True), primary_key=True,
default=uuid4)
    person_id: Mapped[str | None] = mapped_column(UUID(as_uuid=True),
nullable=True)
    username: Mapped[str] = mapped_column(String(120), unique=True,
nullable=False)
    email: Mapped[str] = mapped_column(String(180), unique=True,
nullable=False)
    password_hash: Mapped[str] = mapped_column(String(255), nullable=False)
    status: Mapped[str] = mapped_column(String(20), default="ACTIVE",
nullable=False)
    last_login_at: Mapped[datetime | None] = mapped_column(DateTime)
    must_reset_password: Mapped[bool] = mapped_column(Boolean,
default=False, nullable=False)
    created_at: Mapped[datetime] = mapped_column(DateTime,
server_default=func.now(), nullable=False)
    updated_at: Mapped[datetime] = mapped_column(DateTime,
server_default=func.now(), nullable=False)


class Role(Base):
    __tablename__ = "roles"
    __table_args__ = {"schema": "iam"}

    id: Mapped[str] = mapped_column(UUID(as_uuid=True), primary_key=True,
default=uuid4)
    code: Mapped[str] = mapped_column(String(80), unique=True,
nullable=False)
    name: Mapped[str] = mapped_column(String(120), nullable=False)


class Permission(Base):
    __tablename__ = "permissions"
    __table_args__ = {"schema": "iam"}

    id: Mapped[str] = mapped_column(UUID(as_uuid=True), primary_key=True,
default=uuid4)
    code: Mapped[str] = mapped_column(String(120), unique=True,
nullable=False)
    name: Mapped[str] = mapped_column(String(120), nullable=False)


class UserRole(Base):
    __tablename__ = "user_roles"
    __table_args__ = {"schema": "iam"}

    id: Mapped[str] = mapped_column(UUID(as_uuid=True), primary_key=True,
default=uuid4)
    user_id: Mapped[str] = mapped_column(UUID(as_uuid=True),
```

```
ForeignKey("iam.users.id", ondelete="CASCADE"), nullable=False)
    role_id: Mapped[str] = mapped_column(UUID(as_uuid=True),
ForeignKey("iam.roles.id", ondelete="CASCADE"), nullable=False)
    is_primary: Mapped[bool] = mapped_column(Boolean, default=False,
nullable=False)
    created_at: Mapped[datetime] = mapped_column(DateTime,
server_default=func.now(), nullable=False)


class RolePermission(Base):
    __tablename__ = "role_permissions"
    __table_args__ = {"schema": "iam"}

    id: Mapped[str] = mapped_column(UUID(as_uuid=True), primary_key=True,
default=uuid4)
    role_id: Mapped[str] = mapped_column(UUID(as_uuid=True),
ForeignKey("iam.roles.id", ondelete="CASCADE"), nullable=False)
    permission_id: Mapped[str] = mapped_column(UUID(as_uuid=True),
ForeignKey("iam.permissions.id", ondelete="CASCADE"), nullable=False)
    created_at: Mapped[datetime] = mapped_column(DateTime,
server_default=func.now(), nullable=False)


class UserScopeAssignment(Base):
    __tablename__ = "user_scope_assignments"
    __table_args__ = {"schema": "iam"}

    id: Mapped[str] = mapped_column(UUID(as_uuid=True), primary_key=True,
default=uuid4)
    user_id: Mapped[str] = mapped_column(UUID(as_uuid=True),
ForeignKey("iam.users.id", ondelete="CASCADE"), nullable=False)
    scope_type: Mapped[str] = mapped_column(String(30), nullable=False)
    scope_ref_id: Mapped[str] = mapped_column(UUID(as_uuid=True),
nullable=False)
    created_at: Mapped[datetime] = mapped_column(DateTime,
server_default=func.now(), nullable=False)


class RefreshToken(Base):
    __tablename__ = "refresh_tokens"
    __table_args__ = {"schema": "iam"}

    id: Mapped[str] = mapped_column(UUID(as_uuid=True), primary_key=True,
default=uuid4)
    user_id: Mapped[str] = mapped_column(UUID(as_uuid=True),
ForeignKey("iam.users.id", ondelete="CASCADE"), nullable=False)
    token_hash: Mapped[str] = mapped_column(String(255), nullable=False)
    expires_at: Mapped[datetime] = mapped_column(DateTime, nullable=False)
    revoked_at: Mapped[datetime | None] = mapped_column(DateTime)
    created_at: Mapped[datetime] = mapped_column(DateTime,
server_default=func.now(), nullable=False)
```

`apps/api/app/domain/iam/repository.py`

```
from sqlalchemy import select
from sqlalchemy.orm import Session

from app.domain.iam.models import Permission, RefreshToken, Role,
RolePermission, User, UserRole, UserScopeAssignment


class IAMRepository:
    def __init__(self, db: Session):
        self.db = db

    def get_user_by_username(self, username: str) -> User | None:
        return self.db.execute(select(User).where(User.username ==
username)).scalars().first()

    def get_user_by_id(self, user_id: str) -> User | None:
        return self.db.execute(select(User).where(User.id ==
user_id)).scalars().first()

    def get_role_codes(self, user_id: str) -> list[str]:
        stmt = select(Role.code).join(UserRole, UserRole.role_id ==
Role.id).where(UserRole.user_id == user_id)
        return list(self.db.execute(stmt).scalars().all())

    def get_permission_codes(self, user_id: str) -> list[str]:
        stmt = (
            select(Permission.code)
            .join(RolePermission, RolePermission.permission_id ==
Permission.id)
            .join(Role, Role.id == RolePermission.role_id)
            .join(UserRole, UserRole.role_id == Role.id)
            .where(UserRole.user_id == user_id)
        )
        return list(set(self.db.execute(stmt).scalars().all()))

    def get_scopes(self, user_id: str) -> list[UserScopeAssignment]:
        return
list(self.db.execute(select(UserScopeAssignment).where(UserScopeAssignment.user_id
== user_id)).scalars().all())

    def save_refresh_token(self, entity: RefreshToken) -> RefreshToken:
        self.db.add(entity)
        self.db.flush()
        self.db.refresh(entity)
        return entity
```

Domínio Core

`apps/api/app/domain/core/models.py`

```
from datetime import date, datetime
from uuid import uuid4

from sqlalchemy import Boolean, Date, DateTime, ForeignKey, String, Text,
func
from sqlalchemy.dialects.postgresql import UUID
from sqlalchemy.orm import Mapped, mapped_column

from app.core.database import Base


class Organization(Base):
    __tablename__ = "organizations"
    __table_args__ = {"schema": "core"}

    id: Mapped[str] = mapped_column(UUID(as_uuid=True), primary_key=True,
default=uuid4)
    type: Mapped[str] = mapped_column(String(30), nullable=False)
    name: Mapped[str] = mapped_column(String(255), nullable=False)
    legal_name: Mapped[str | None] = mapped_column(String(255))
    document_number: Mapped[str | None] = mapped_column(String(30))
    parent_organization_id: Mapped[str | None] =
mapped_column(UUID(as_uuid=True), ForeignKey("core.organizations.id"))
    active: Mapped[bool] = mapped_column(Boolean, default=True,
nullable=False)
    created_at: Mapped[datetime] = mapped_column(DateTime,
server_default=func.now(), nullable=False)
    updated_at: Mapped[datetime] = mapped_column(DateTime,
server_default=func.now(), nullable=False)


class Person(Base):
    __tablename__ = "persons"
    __table_args__ = {"schema": "core"}

    id: Mapped[str] = mapped_column(UUID(as_uuid=True), primary_key=True,
default=uuid4)
    full_name: Mapped[str] = mapped_column(String(255), nullable=False)
    social_name: Mapped[str | None] = mapped_column(String(255))
    cpf: Mapped[str | None] = mapped_column(String(20))
    birth_date: Mapped[date | None] = mapped_column(Date)
    phone: Mapped[str | None] = mapped_column(String(30))
    secondary_phone: Mapped[str | None] = mapped_column(String(30))
    email: Mapped[str | None] = mapped_column(String(180))
    gender: Mapped[str | None] = mapped_column(String(30))
    notes: Mapped[str | None] = mapped_column(Text)
```

```
    created_at: Mapped[datetime] = mapped_column(DateTime,
server_default=func.now(), nullable=False)
    updated_at: Mapped[datetime] = mapped_column(DateTime,
server_default=func.now(), nullable=False)
```

`apps/api/app/domain/core/schemas.py`

```
from datetime import date
from uuid import UUID

from pydantic import BaseModel


class PersonCreate(BaseModel):
    full_name: str
    social_name: str | None = None
    cpf: str | None = None
    birth_date: date | None = None
    phone: str | None = None
    secondary_phone: str | None = None
    email: str | None = None
    gender: str | None = None
    notes: str | None = None


class PersonOut(BaseModel):
    id: UUID
    full_name: str
    social_name: str | None = None
    cpf: str | None = None
    birth_date: date | None = None
    phone: str | None = None
    email: str | None = None

    model_config = {"from_attributes": True}
```

`apps/api/app/domain/core/repository.py`

```
from sqlalchemy import select
from sqlalchemy.orm import Session

from app.domain.core.models import Person


class CoreRepository:
    def __init__(self, db: Session):
        self.db = db

    def list_persons(self):
```

```
        return self.db.execute(select(Person)).scalars().all()

    def create_person(self, entity: Person) -> Person:
        self.db.add(entity)
        self.db.flush()
        self.db.refresh(entity)
        return entity
```

`apps/api/app/domain/core/service.py`

```
from app.core.audit import audited_mutation
from app.domain.core.models import Person
from app.domain.core.repository import CoreRepository
from app.domain.core.schemas import PersonCreate


class CoreService:
    def __init__(self, repo: CoreRepository):
        self.repo = repo

    def list_persons(self):
        return self.repo.list_persons()

    @audited_mutation(action="CREATE", entity_schema="core",
entity_name="persons")
    def create_person(self, payload: PersonCreate, db=None,
current_user=None):
        entity = Person(**payload.model_dump())
        return self.repo.create_person(entity)
```

Domínio Territory

`apps/api/app/domain/territory/models.py`

```
from datetime import datetime
from uuid import uuid4

from sqlalchemy import DateTime, ForeignKey, Numeric, String, Text, func
from sqlalchemy.dialects.postgresql import UUID
from sqlalchemy.orm import Mapped, mapped_column

from app.core.database import Base


class ContactCapture(Base):
    __tablename__ = "contacts_capture"
    __table_args__ = {"schema": "territory"}
```

```
    id: Mapped[str] = mapped_column(UUID(as_uuid=True), primary_key=True,
default=uuid4)
    captured_by_user_id: Mapped[str] = mapped_column(UUID(as_uuid=True),
ForeignKey("iam.users.id"), nullable=False)
    organization_id: Mapped[str | None] = mapped_column(UUID(as_uuid=True),
ForeignKey("core.organizations.id"))
    vereador_id: Mapped[str | None] = mapped_column(UUID(as_uuid=True),
ForeignKey("core.vereadores.id"))
    team_id: Mapped[str | None] = mapped_column(UUID(as_uuid=True),
ForeignKey("core.teams.id"))
    person_id: Mapped[str | None] = mapped_column(UUID(as_uuid=True),
ForeignKey("core.persons.id"))
    origin: Mapped[str] = mapped_column(String(30), nullable=False)
    classification: Mapped[str] = mapped_column(String(30), nullable=False)
    full_name: Mapped[str] = mapped_column(String(255), nullable=False)
    phone: Mapped[str | None] = mapped_column(String(30))
    district: Mapped[str | None] = mapped_column(String(120))
    notes: Mapped[str | None] = mapped_column(Text)
    priority_level: Mapped[str | None] = mapped_column(String(20))
    capture_status: Mapped[str] = mapped_column(String(30), default="NEW",
nullable=False)
    latitude: Mapped[float | None] = mapped_column(Numeric(10, 7))
    longitude: Mapped[float | None] = mapped_column(Numeric(10, 7))
    created_at: Mapped[datetime] = mapped_column(DateTime,
server_default=func.now(), nullable=False)
    updated_at: Mapped[datetime] = mapped_column(DateTime,
server_default=func.now(), nullable=False)
```

`apps/api/app/domain/territory/schemas.py`

```
from uuid import UUID

from pydantic import BaseModel


class ContactCaptureCreate(BaseModel):
    origin: str
    classification: str
    full_name: str
    phone: str | None = None
    district: str | None = None
    notes: str | None = None
    vereador_id: UUID | None = None
    team_id: UUID | None = None
    latitude: float | None = None
    longitude: float | None = None


class ContactCaptureOut(BaseModel):
```

```
    id: UUID
    origin: str
    classification: str
    full_name: str
    phone: str | None = None
    district: str | None = None
    notes: str | None = None
    capture_status: str

    model_config = {"from_attributes": True}
```

apps/api/app/domain/territory/repository.py

```
from sqlalchemy import select
from sqlalchemy.orm import Session

from app.domain.territory.models import ContactCapture


class TerritoryRepository:
    def __init__(self, db: Session):
        self.db = db

    def list_captures(self):
        return self.db.execute(select(ContactCapture)).scalars().all()

    def create_capture(self, entity: ContactCapture) -> ContactCapture:
        self.db.add(entity)
        self.db.flush()
        self.db.refresh(entity)
        return entity
```

apps/api/app/domain/territory/service.py

```
from app.core.audit import audited_mutation
from app.domain.territory.models import ContactCapture
from app.domain.territory.repository import TerritoryRepository
from app.domain.territory.schemas import ContactCaptureCreate


class TerritoryService:
    def __init__(self, repo: TerritoryRepository):
        self.repo = repo

    def list_captures(self):
        return self.repo.list_captures()

    @audited_mutation(action="CREATE", entity_schema="territory",
entity_name="contacts_capture")
```

```
    def create_capture(self, captured_by_user_id, payload:
ContactCaptureCreate, db=None, current_user=None):
        entity = ContactCapture(
            captured_by_user_id=captured_by_user_id,
            vereador_id=payload.vereador_id,
            team_id=payload.team_id,
            origin=payload.origin,
            classification=payload.classification,
            full_name=payload.full_name,
            phone=payload.phone,
            district=payload.district,
            notes=payload.notes,
            latitude=payload.latitude,
            longitude=payload.longitude,
        )
        return self.repo.create_capture(entity)
```

Domínio Polo

`apps/api/app/domain/polo/models.py`

```
from datetime import date, datetime
from uuid import uuid4

from sqlalchemy import Boolean, Date, DateTime, ForeignKey, String, Text,
func
from sqlalchemy.dialects.postgresql import UUID
from sqlalchemy.orm import Mapped, mapped_column

from app.core.database import Base


class PoloUnit(Base):
    __tablename__ = "units"
    __table_args__ = {"schema": "polo"}

    id: Mapped[str] = mapped_column(UUID(as_uuid=True), primary_key=True,
default=uuid4)
    organization_id: Mapped[str] = mapped_column(UUID(as_uuid=True),
ForeignKey("core.organizations.id"), unique=True)
    code: Mapped[str | None] = mapped_column(String(50))
    address_label: Mapped[str | None] = mapped_column(String(255))
    active: Mapped[bool] = mapped_column(Boolean, default=True,
nullable=False)
    created_at: Mapped[datetime] = mapped_column(DateTime,
server_default=func.now(), nullable=False)
```

```
class PoloBeneficiario(Base):
    __tablename__ = "beneficiarios"
    __table_args__ = {"schema": "polo"}

    id: Mapped[str] = mapped_column(UUID(as_uuid=True), primary_key=True,
default=uuid4)
    polo_id: Mapped[str] = mapped_column(UUID(as_uuid=True),
ForeignKey("polo.units.id"), nullable=False)
    person_id: Mapped[str] = mapped_column(UUID(as_uuid=True),
ForeignKey("core.persons.id"), nullable=False)
    source_capture_id: Mapped[str | None] =
mapped_column(UUID(as_uuid=True),
ForeignKey("territory.contacts_capture.id"))
    status: Mapped[str] = mapped_column(String(30),
default="PRE_CADASTRADO", nullable=False)
    admitted_at: Mapped[datetime | None] = mapped_column(DateTime)
    discharged_at: Mapped[datetime | None] = mapped_column(DateTime)
    created_at: Mapped[datetime] = mapped_column(DateTime,
server_default=func.now(), nullable=False)


class Frequencia(Base):
    __tablename__ = "frequencias"
    __table_args__ = {"schema": "polo"}

    id: Mapped[str] = mapped_column(UUID(as_uuid=True), primary_key=True,
default=uuid4)
    beneficiario_id: Mapped[str] = mapped_column(UUID(as_uuid=True),
ForeignKey("polo.beneficiarios.id"), nullable=False)
    modalidade_id: Mapped[str | None] = mapped_column(UUID(as_uuid=True),
ForeignKey("polo.modalidades.id"))
    registered_by_user_id: Mapped[str] = mapped_column(UUID(as_uuid=True),
ForeignKey("iam.users.id"), nullable=False)
    activity_date: Mapped[date] = mapped_column(Date, nullable=False)
    present: Mapped[bool] = mapped_column(Boolean, nullable=False)
    notes: Mapped[str | None] = mapped_column(Text)
    created_at: Mapped[datetime] = mapped_column(DateTime,
server_default=func.now(), nullable=False)
```

`apps/api/app/domain/polo/schemas.py`

```
from datetime import date
from uuid import UUID

from pydantic import BaseModel, Field


class PoloBeneficiarioCreate(BaseModel):
    person_id: UUID
    source_capture_id: UUID | None = None
```

```
    status: str = "PRE_CADASTRADO"


class PoloBeneficiarioOut(BaseModel):
    id: UUID
    polo_id: UUID
    person_id: UUID
    source_capture_id: UUID | None = None
    status: str

    model_config = {"from_attributes": True}


class AttendanceCreate(BaseModel):
    beneficiario_id: UUID
    modalidade_id: UUID | None = None
    activity_date: date
    present: bool
    notes: str | None = Field(default=None, max_length=2000)


class AttendanceOut(BaseModel):
    id: UUID
    beneficiario_id: UUID
    modalidade_id: UUID | None = None
    activity_date: date
    present: bool
    notes: str | None = None

    model_config = {"from_attributes": True}
```

`apps/api/app/domain/polo/repository.py`

```
from sqlalchemy import select
from sqlalchemy.orm import Session

from app.domain.polo.models import Frequencia, PoloBeneficiario


class PoloRepository:
    def __init__(self, db: Session):
        self.db = db

    def list_beneficiarios(self, polo_id):
        return
self.db.execute(select(PoloBeneficiario).where(PoloBeneficiario.polo_id ==
polo_id)).scalars().all()

    def create_beneficiario(self, entity: PoloBeneficiario) ->
PoloBeneficiario:
```

```
        self.db.add(entity)
        self.db.flush()
        self.db.refresh(entity)
        return entity

    def create_frequencia(self, entity: Frequencia) -> Frequencia:
        self.db.add(entity)
        self.db.flush()
        self.db.refresh(entity)
        return entity
```

`apps/api/app/domain/polo/service.py`

```
from app.core.audit import audited_mutation
from app.domain.polo.models import Frequencia, PoloBeneficiario
from app.domain.polo.repository import PoloRepository
from app.domain.polo.schemas import AttendanceCreate, PoloBeneficiarioCreate


class PoloService:
    def __init__(self, repo: PoloRepository):
        self.repo = repo

    def list_beneficiarios(self, polo_id):
        return self.repo.list_beneficiarios(polo_id)

    @audited_mutation(action="CREATE", entity_schema="polo",
entity_name="beneficiarios")
    def create_beneficiario(self, polo_id, payload: PoloBeneficiarioCreate,
db=None, current_user=None):
        entity = PoloBeneficiario(
            polo_id=polo_id,
            person_id=payload.person_id,
            source_capture_id=payload.source_capture_id,
            status=payload.status,
        )
        return self.repo.create_beneficiario(entity)

    @audited_mutation(action="CREATE", entity_schema="polo",
entity_name="frequencias")
    def register_attendance(self, registered_by_user_id, payload:
AttendanceCreate, db=None, current_user=None):
        entity = Frequencia(
            beneficiario_id=payload.beneficiario_id,
            modalidade_id=payload.modalidade_id,
            registered_by_user_id=registered_by_user_id,
            activity_date=payload.activity_date,
            present=payload.present,
            notes=payload.notes,
```

```
        )
        return self.repo.create_frequencia(entity)
```

Domínio Workflow

`apps/api/app/domain/workflow/models.py`

```
from datetime import datetime
from uuid import uuid4

from sqlalchemy import DateTime, ForeignKey, String, Text, func
from sqlalchemy.dialects.postgresql import UUID
from sqlalchemy.orm import Mapped, mapped_column

from app.core.database import Base


class Task(Base):
    __tablename__ = "tasks"
    __table_args__ = {"schema": "workflow"}

    id: Mapped[str] = mapped_column(UUID(as_uuid=True), primary_key=True,
default=uuid4)
    organization_id: Mapped[str | None] = mapped_column(UUID(as_uuid=True),
ForeignKey("core.organizations.id"))
    vereador_id: Mapped[str | None] = mapped_column(UUID(as_uuid=True),
ForeignKey("core.vereadores.id"))
    polo_id: Mapped[str | None] = mapped_column(UUID(as_uuid=True),
ForeignKey("polo.units.id"))
    person_id: Mapped[str | None] = mapped_column(UUID(as_uuid=True),
ForeignKey("core.persons.id"))
    demand_id: Mapped[str | None] = mapped_column(UUID(as_uuid=True),
ForeignKey("territory.demands.id"))
    assigned_to_user_id: Mapped[str | None] =
mapped_column(UUID(as_uuid=True), ForeignKey("iam.users.id"))
    created_by_user_id: Mapped[str] = mapped_column(UUID(as_uuid=True),
ForeignKey("iam.users.id"), nullable=False)
    task_type: Mapped[str] = mapped_column(String(50), nullable=False)
    title: Mapped[str] = mapped_column(String(255), nullable=False)
    description: Mapped[str | None] = mapped_column(Text)
    priority: Mapped[str] = mapped_column(String(20), default="MEDIUM",
nullable=False)
    status: Mapped[str] = mapped_column(String(20), default="OPEN",
nullable=False)
    due_at: Mapped[datetime | None] = mapped_column(DateTime)
    completed_at: Mapped[datetime | None] = mapped_column(DateTime)
    created_at: Mapped[datetime] = mapped_column(DateTime,
server_default=func.now(), nullable=False)
```

```
    updated_at: Mapped[datetime] = mapped_column(DateTime,
server_default=func.now(), nullable=False)
```

apps/api/app/domain/workflow/schemas.py

```
from datetime import datetime
from uuid import UUID

from pydantic import BaseModel


class TaskCreate(BaseModel):
    organization_id: UUID | None = None
    vereador_id: UUID | None = None
    polo_id: UUID | None = None
    person_id: UUID | None = None
    demand_id: UUID | None = None
    assigned_to_user_id: UUID | None = None
    task_type: str
    title: str
    description: str | None = None
    priority: str = "MEDIUM"
    due_at: datetime | None = None


class TaskOut(BaseModel):
    id: UUID
    task_type: str
    title: str
    description: str | None = None
    priority: str
    status: str
    due_at: datetime | None = None

    model_config = {"from_attributes": True}
```

apps/api/app/domain/workflow/repository.py

```
from sqlalchemy import select
from sqlalchemy.orm import Session

from app.domain.workflow.models import Task


class WorkflowRepository:
    def __init__(self, db: Session):
        self.db = db

    def list_tasks(self):
```

```
        return self.db.execute(select(Task)).scalars().all()

    def create_task(self, entity: Task) -> Task:
        self.db.add(entity)
        self.db.flush()
        self.db.refresh(entity)
        return entity
```

`apps/api/app/domain/workflow/service.py`

```
from app.core.audit import audited_mutation
from app.domain.workflow.models import Task
from app.domain.workflow.repository import WorkflowRepository
from app.domain.workflow.schemas import TaskCreate


class WorkflowService:
    def __init__(self, repo: WorkflowRepository):
        self.repo = repo

    def list_tasks(self):
        return self.repo.list_tasks()

    @audited_mutation(action="CREATE", entity_schema="workflow",
entity_name="tasks")
    def create_task(self, created_by_user_id, payload: TaskCreate, db=None,
current_user=None):
        entity = Task(
            organization_id=payload.organization_id,
            vereador_id=payload.vereador_id,
            polo_id=payload.polo_id,
            person_id=payload.person_id,
            demand_id=payload.demand_id,
            assigned_to_user_id=payload.assigned_to_user_id,
            created_by_user_id=created_by_user_id,
            task_type=payload.task_type,
            title=payload.title,
            description=payload.description,
            priority=payload.priority,
            due_at=payload.due_at,
        )
        return self.repo.create_task(entity)
```

Domínio Analytics

`apps/api/app/domain/analytics/repository.py`

```
from sqlalchemy import text
from sqlalchemy.orm import Session


class AnalyticsRepository:
    def __init__(self, db: Session):
        self.db = db

    def get_vereador_dashboard(self, vereador_id):
        stmt = text(
            """
            select vereador_id, total_captures, total_beneficiarios,
open_demands, open_tasks
            from analytics.mv_vereador_dashboard
            where vereador_id = :vereador_id
            """
        )
        row = self.db.execute(stmt, {"vereador_id":
vereador_id}).mappings().first()
        return dict(row) if row else None
```

`apps/api/app/domain/analytics/service.py`

```
from app.domain.analytics.repository import AnalyticsRepository


class AnalyticsService:
    def __init__(self, repo: AnalyticsRepository):
        self.repo = repo

    def get_vereador_dashboard(self, vereador_id):
        return self.repo.get_vereador_dashboard(vereador_id)
```

Scripts

`apps/api/scripts/seed_initial_data.py`

```
print("Carregue aqui os seeds iniciais de roles, permissions e
role_permissions.")
```

`apps/api/scripts/bootstrap_admin.py`

```
import uuid

from app.core.database import SessionLocal
from app.core.security import hash_password
from app.domain.iam.models import User


def main():
    db = SessionLocal()
    try:
        exists = db.query(User).filter(User.username == "admin").first()
        if exists:
            print("admin já existe")
            return
        user = User(
            id=uuid.uuid4(),
            username="admin",
            email="admin@revisa.local",
            password_hash=hash_password("Admin@123"),
            status="ACTIVE",
        )
        db.add(user)
        db.commit()
        print("admin criado")
    finally:
        db.close()


if __name__ == "__main__":
    main()
```

Seeds e migrations

`database/seeds/permissions_seed.sql`

```
insert into iam.permissions (id, code, name) values
  (gen_random_uuid(), 'auth.login', 'Realizar login'),
  (gen_random_uuid(), 'user.read', 'Consultar usuários'),
  (gen_random_uuid(), 'user.create', 'Criar usuários'),
  (gen_random_uuid(), 'user.update', 'Atualizar usuários'),
  (gen_random_uuid(), 'user.manage_roles', 'Gerir papéis de usuários'),
  (gen_random_uuid(), 'user.manage_scopes', 'Gerir escopos de usuários'),
  (gen_random_uuid(), 'organization.read', 'Consultar organizações'),
  (gen_random_uuid(), 'organization.create', 'Criar organizações'),
  (gen_random_uuid(), 'organization.update', 'Atualizar organizações'),
```

```
  (gen_random_uuid(), 'vereador.read', 'Consultar vereadores'),
  (gen_random_uuid(), 'vereador.create', 'Criar vereadores'),
  (gen_random_uuid(), 'vereador.update', 'Atualizar vereadores'),
  (gen_random_uuid(), 'team.read', 'Consultar equipes'),
  (gen_random_uuid(), 'team.create', 'Criar equipes'),
  (gen_random_uuid(), 'team.update', 'Atualizar equipes'),
  (gen_random_uuid(), 'person.read', 'Consultar pessoas'),
  (gen_random_uuid(), 'person.create', 'Criar pessoas'),
  (gen_random_uuid(), 'person.update', 'Atualizar pessoas'),
  (gen_random_uuid(), 'person.link', 'Vincular pessoa a contexto'),
  (gen_random_uuid(), 'consent.read', 'Consultar consentimentos'),
  (gen_random_uuid(), 'consent.create', 'Criar consentimentos'),
  (gen_random_uuid(), 'consent.revoke', 'Revogar consentimentos'),
  (gen_random_uuid(), 'capture.read', 'Consultar captações'),
  (gen_random_uuid(), 'capture.create', 'Criar captações'),
  (gen_random_uuid(), 'capture.classify', 'Classificar captações'),
  (gen_random_uuid(), 'capture.forward', 'Encaminhar captações'),
  (gen_random_uuid(), 'capture.convert', 'Converter captação em
beneficiário'),
  (gen_random_uuid(), 'polo.read', 'Consultar polos'),
  (gen_random_uuid(), 'polo.create', 'Criar polos'),
  (gen_random_uuid(), 'polo.update', 'Atualizar polos'),
  (gen_random_uuid(), 'polo.manage_beneficiary', 'Gerir beneficiários do
polo'),
  (gen_random_uuid(), 'modality.read', 'Consultar modalidades'),
  (gen_random_uuid(), 'modality.create', 'Criar modalidades'),
  (gen_random_uuid(), 'modality.update', 'Atualizar modalidades'),
  (gen_random_uuid(), 'attendance.read', 'Consultar frequências'),
  (gen_random_uuid(), 'attendance.create', 'Registrar frequências'),
  (gen_random_uuid(), 'occurrence.read', 'Consultar ocorrências'),
  (gen_random_uuid(), 'occurrence.create', 'Registrar ocorrências'),
  (gen_random_uuid(), 'daily_log.read', 'Consultar diário do polo'),
  (gen_random_uuid(), 'daily_log.create', 'Criar diário do polo'),
  (gen_random_uuid(), 'purchase_request.read', 'Consultar pedidos de
compra'),
  (gen_random_uuid(), 'purchase_request.create', 'Criar pedidos de compra'),
  (gen_random_uuid(), 'cabinet.read', 'Consultar gabinete'),
  (gen_random_uuid(), 'cabinet.action.read', 'Consultar ações do gabinete'),
  (gen_random_uuid(), 'cabinet.action.create', 'Criar ações do gabinete'),
  (gen_random_uuid(), 'task.read', 'Consultar tarefas'),
  (gen_random_uuid(), 'task.create', 'Criar tarefas'),
  (gen_random_uuid(), 'task.update', 'Atualizar tarefas'),
  (gen_random_uuid(), 'task.complete', 'Concluir tarefas'),
  (gen_random_uuid(), 'demand.read', 'Consultar demandas'),
  (gen_random_uuid(), 'demand.create', 'Criar demandas'),
  (gen_random_uuid(), 'demand.update', 'Atualizar demandas'),
  (gen_random_uuid(), 'demand.assign', 'Atribuir demandas'),
  (gen_random_uuid(), 'event.read', 'Consultar eventos e atividades'),
  (gen_random_uuid(), 'event.create', 'Criar eventos e atividades'),
  (gen_random_uuid(), 'event.update', 'Atualizar eventos e atividades'),
  (gen_random_uuid(), 'dashboard.admin.read', 'Consultar dashboard
```

```
administrativo'),
  (gen_random_uuid(), 'dashboard.vereador.read', 'Consultar dashboard do
vereador'),
  (gen_random_uuid(), 'dashboard.polo.read', 'Consultar dashboard do polo'),
  (gen_random_uuid(), 'dashboard.cabinet.read', 'Consultar dashboard do
gabinete'),
  (gen_random_uuid(), 'geo.read', 'Consultar camadas geográficas'),
  (gen_random_uuid(), 'geo.manage', 'Gerir camadas geográficas'),
  (gen_random_uuid(), 'report.read', 'Consultar relatórios'),
  (gen_random_uuid(), 'report.export', 'Exportar relatórios'),
  (gen_random_uuid(), 'audit.read', 'Consultar auditoria'),
  (gen_random_uuid(), 'privacy.read', 'Consultar solicitações de
privacidade'),
  (gen_random_uuid(), 'privacy.process', 'Processar solicitações de
privacidade');
```

`apps/api/alembic/versions/0001_create_iam_schema.py` **até** `0008_create_analytics_schema.py`

Use exatamente os arquivos de migration já fechados no pacote operacional anterior, sem alteração estrutural.

Testes de integração

`apps/api/app/tests/conftest.py`

```
import os

import pytest
from fastapi.testclient import TestClient
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker

from app.core.database import Base, get_db
from app.main import app

TEST_DATABASE_URL = os.getenv("TEST_DATABASE_URL", "postgresql+psycopg://
revisa:revisa@localhost:5432/revisa_test")

engine = create_engine(TEST_DATABASE_URL, future=True)
TestingSessionLocal = sessionmaker(bind=engine, autoflush=False,
autocommit=False, future=True)


@pytest.fixture(scope="session", autouse=True)
def create_test_db():
    Base.metadata.create_all(bind=engine)
    yield
```

```
    Base.metadata.drop_all(bind=engine)


@pytest.fixture()
def db_session():
    connection = engine.connect()
    transaction = connection.begin()
    session = TestingSessionLocal(bind=connection)
    yield session
    session.close()
    transaction.rollback()
    connection.close()


@pytest.fixture()
def client(db_session):
    def override_get_db():
        yield db_session

    app.dependency_overrides[get_db] = override_get_db
    with TestClient(app) as c:
        yield c
    app.dependency_overrides.clear()
```

`apps/api/app/tests/integration/test_auth_login.py`

```
from app.core.security import hash_password
from app.domain.iam.models import User


def test_login_success(client, db_session):
    user = User(
        username="admin",
        email="admin@revisa.local",
        password_hash=hash_password("123456"),
        status="ACTIVE",
    )
    db_session.add(user)
    db_session.commit()

    response = client.post("/api/v1/auth/login", json={"username": "admin",
"password": "123456"})
    assert response.status_code == 200
    assert "access_token" in response.json()
```

`apps/api/app/tests/integration/test_persons.py`

```
def test_list_persons_requires_auth(client):
    response = client.get("/api/v1/persons")
    assert response.status_code in (401, 403)
```

`apps/api/app/tests/integration/test_contacts_capture.py`

```
def test_create_capture_requires_auth(client):
    response = client.post(
        "/api/v1/contacts-capture",
        json={"origin": "MOBILE", "classification": "CIDADAO", "full_name":
"Maria da Silva"},
    )
    assert response.status_code in (401, 403)
```

`apps/api/app/tests/integration/test_polos.py`

```
def test_list_beneficiarios_requires_auth(client):
    response = client.get("/api/v1/polos/
11111111-1111-1111-1111-111111111111/beneficiarios")
    assert response.status_code in (401, 403)
```

`apps/api/app/tests/integration/test_tasks.py`

```
def test_list_tasks_requires_auth(client):
    response = client.get("/api/v1/tasks")
    assert response.status_code in (401, 403)
```

`apps/api/app/tests/integration/test_dashboards.py`

```
def test_dashboard_requires_auth(client):
    response = client.get("/api/v1/vereadores/
11111111-1111-1111-1111-111111111111/dashboard")
    assert response.status_code in (401, 403)
```

Pipeline CI com PostgreSQL real

`.github/workflows/api-ci.yml`

```
name: API CI

on:
```

```yaml
  push:
    paths:
      - 'apps/api/**'
      - '.github/workflows/api-ci.yml'
  pull_request:
    paths:
      - 'apps/api/**'
      - '.github/workflows/api-ci.yml'

jobs:
  test:
    runs-on: ubuntu-latest

    services:
      postgres:
        image: postgres:16
        env:
          POSTGRES_DB: revisa_test
          POSTGRES_USER: revisa
          POSTGRES_PASSWORD: revisa
        ports:
          - 5432:5432
        options: >-
          --health-cmd "pg_isready -U revisa -d revisa_test"
          --health-interval 10s
          --health-timeout 5s
          --health-retries 10

    env:
      DATABASE_URL: postgresql+psycopg://revisa:revisa@localhost:5432/
revisa_test
      TEST_DATABASE_URL: postgresql+psycopg://revisa:revisa@localhost:5432/
revisa_test
      JWT_SECRET_KEY: test_secret_key
      JWT_REFRESH_SECRET_KEY: test_refresh_secret_key
      API_V1_PREFIX: /api/v1

    defaults:
      run:
        working-directory: apps/api

    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.11'

      - name: Install dependencies
```

```
        run: |
          python -m pip install --upgrade pip
          pip install -e .

      - name: Run Alembic migrations
        run: alembic upgrade head

      - name: Seed permissions
        run: |
          python - <<'PY'
          print('seed step placeholder: execute seed_initial_data.py or raw
SQL bootstrap')
          PY

      - name: Run tests
        run: pytest -q
```

Fechamento operacional

Este pacote deixa o backend em ponto de materialização rápida no monorepo.

O que ainda precisa ser copiado do material anterior sem reescrita aqui:

- conteúdo completo das 8 migrations Alembic
- conteúdo completo do `openapi.yaml`
- seeds completos de roles e role_permissions

Para a equipe, a sequência de execução é: 1. colar os arquivos acima no monorepo 2. colar as migrations já fechadas nas versões correspondentes 3. aplicar seeds 4. subir PostgreSQL local 5. rodar `alembic upgrade head` 6. rodar `python scripts/bootstrap_admin.py` 7. subir `uvicorn app.main:app --reload` 8. validar CI no GitHub Actions ```